

Spring

introduzione

Introduzione

- Spring framework Fornisce un ambiente flessibile e completamente configurabile.
- Diversi progetti Java che sfruttavano il framework Spring avevano una configurazione dell'infrastruttura molto simile.
- Si stavano creando delle vere e proprie convenzioni per quanto riguarda la configurazione di un'applicazione Spring.
- Da qui nasce Spring Boot.

Spring boot

introduzione

Introduzione

- Spring Boot è un progetto open source che fa parte del framework Spring.
- Sviluppato per semplificare la creazione di applicazioni stand-alone, production-grade basate sul framework Spring.
- Stand-alone: Applicazione che non deve essere «distribuita» su un web server. Il web server è incorporato.
- Production-grade: Applicazioni robuste che possono essere mandate in produzione.

Introduzione

- Spring Boot si basa sull'approccio «Convention over configuration».
 - Se l'applicazione segue le convenzioni stabilite, non ci sarà la necessità di ulteriori file di configurazione.
 - Saranno necessari file di configurazione se il comportamento dell'app si discosta dalla convenzione.
- Tale concetto è usato nei framework per:
 - Ridurre il numero di decisioni che uno sviluppatore deve prendere.
 - Mantenere flessibilità

Vantaggi di Spring Boot

- Riduce il tempo necessario per lo sviluppo e incrementa la produttività:
- Consente agli sviluppatori di concentrarsi sullo sviluppo della business logic dell'applicazione, invece di occupare troppo tempo per configurare l'infrastruttura dell'applicazione.
- offre un approccio più semplice e rapido per impostare, configurare ed eseguire le app.
- Elimina la complessità della configurazione necessaria per app basate su Spring.

Vantaggi di Spring Boot

- Fornisce delle funzionalità «non funzionali» molto comuni alle applicazioni:
 - Server incorporati, sicurezza, metriche, controllo salute dell'applicazione e configurazioni esternalizzate .
- Riduce la necessità di scrivere codice «boilerplate» e configurazioni XML:
 - Non dobbiamo generare codice che si ripete.
 - Non è necessario aggiungere delle configurazioni XML.

Vantaggi di Spring Boot

- Packaging semplificato:
 - Tutto quello di cui l'app ha bisogno (codice e dipendenze) è presente dentro un JAR che può essere mandato in esecuzione direttamente con il comando «java -jar <name.jar>»
 - Non più tanti JAR singoli per mandare in produzione un'applicazione in sicurezza.

Vantaggi di Spring Boot

- Integrazione delle app Spring Boot con altri progetti Spring framework:
 - Le applicazioni Spring boot si integrano perfettamente con altri progetti dell'ecosistema Spring framework come Spring Data, Spring Security, Spring Cloud ecc..

Vantaggi di Spring Boot

- Gestione delle dipendenze semplificato:
 - In passato, per sviluppare un'applicazione Spring framework gli sviluppatori occupavano tanto tempo nella gestione delle dipendenze(tanti JAR) e delle versioni di esse.
 - Adesso possiamo usare gli «Starters»

Configurazione automatica

- Spring Boot è composto da un insieme di macro librerie che consentono di automatizzare la fase di configurazione dell'app.
- Queste macro librerie si chiamano «starters».
- Ogni starter che usiamo nel nostro progetto contiene un insieme di JAR.
- Gli starter sono opinionati, cioè si basano sull'opinione dei creatori di Spring di ciò che pensano possa servire al tuo progetto.

Configurazione automatica

- Esempio, se inserisco lo starter-data-jpa nel progetto, allora significa che nel classpath verranno configurati N JAR che fanno parte dello starter-data-jpa. Tra cui hibernate, transactions e molti altri JAR.
- Spring boot capirà quali bean creare e come configurarli tramite lo scanning dei JAR presenti nel classpath.
- La configurazione del nostro progetto Spring Boot dipende da quali starters abbiamo scelto come dipendenze.

Come utilizzare Spring Boot

- Possiamo creare un progetto Spring Boot tramite:
 - Applicativo web chiamato Spring Initializr.
 - IDE.
 - Command line interface di Spring Boot.
 - Tool di build esterni come ad esempio Maven o Gradle

Spring Initializr

- Sono dei web service REST con un'interfaccia grafica che si occupano di generare il progetto in base alle nostre specifiche.
- Interfaccia grafica raggiungibile all'indirizzo: <https://start.spring.io/>
- Dobbiamo fornire le specifiche del nostro progetto:
 - Tipo di progetto(Maven o Gradle)
 - Linguaggio(Java o altri)
 - Versione di Spring Boot
 - Project Metadata(groupid, artifact, packaging ecc..)
 - Dependencies(Selezionare gli starters di cui abbiamo bisogno)

Spring Initializr



The Spring Initializr logo, consisting of a green hexagon with a white leaf-like shape inside, followed by the text 'spring initializr' in a white, lowercase, sans-serif font.

Project
☒ Gradle - Groovy ☐ Gradle - Kotlin ☐ Maven
Spring Boot
☐ 3.1.0 (SNAPSHOT) ☐ 3.1.0 (M1) ☐ 3.0.5 (SNAPSHOT) ☒ 3.0.4
☐ 2.7.10 (SNAPSHOT) ☐ 2.7.9
Project Metadata
Group
Artifact
Name
Description
Package name
Packaging ☒ Jar ☐ War
Java ☐ 19 ☒ 17 ☐ 11 ☐ 8

Language
☒ Java ☐ Kotlin ☐ Groovy

Dependencies

ADD DEPENDENCIES... CTRL + B

No dependency selected

GENERATE CTRL + G

EXPLORE CTRL + SPACE

SHARE...



IDE (Spring tool suite)



New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:



New Spring Starter Project Dependencies

Spring Boot Version:

Available:

Selected:

- Developer Tools
- Google Cloud Platform
- I/O
- Messaging
- Microsoft Azure
- NoSQL
- Observability
- Ops
- SQL
- Security
- Spring Cloud
- Spring Cloud Circuit Breaker
- Spring Cloud Config
- Spring Cloud Discovery
- Spring Cloud Messaging
- Spring Cloud Routing
- Spring Cloud Tools
- Template Engines
- Testing
- VMware Tanzu Application Service



Starters

➤ spring-boot-starter-parent:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.4.4</version>
  <relativePath/>
</parent>
```

- Questo starter serve per specificare la versione di Spring Boot.
- Specificando la versione nel tag <parent> stiamo comunicando al pom «padre» che tutti gli starter che verranno inseriti nel pom «figlio», ovvero nel nostro pom, saranno della stessa versione specificata per il parent.

Starters

➤ spring-boot-starter:

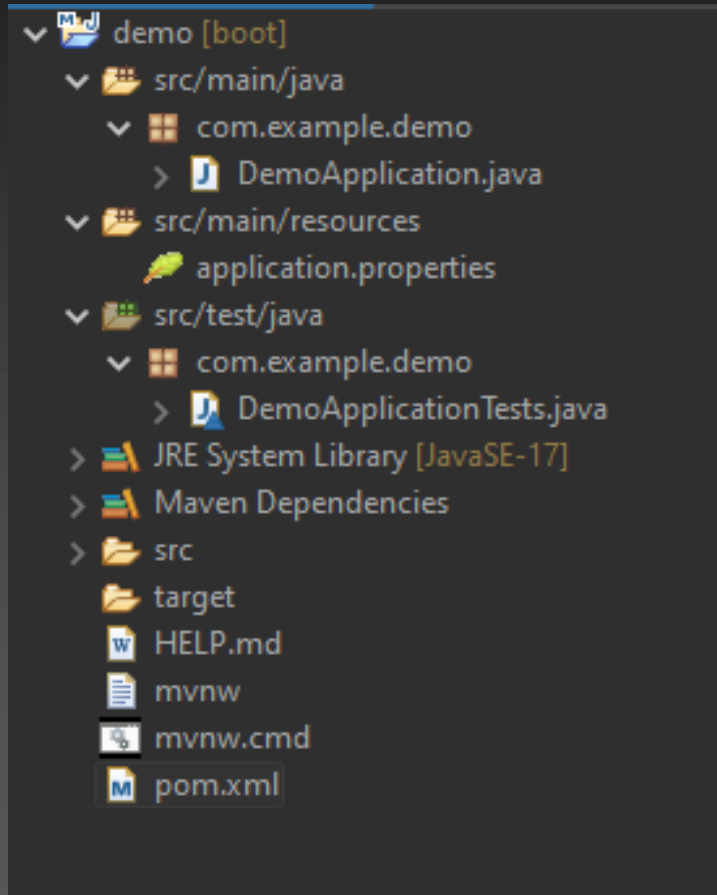
```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter</artifactId>  
</dependency>
```

- Questo starter contiene tutti i JAR necessari per creare un progetto Spring boot.
- Contiene i JAR spring-core, spring-context ecc.. Tutti JAR che in passato avremmo dovuto inserire singolarmente con applicazioni basate su Spring framework.
- Altri starters come «starter-web» o «starter-data-jpa» includono lo starter «spring-boot-starter» al loro interno. Per questo motivo spesso viene omissso.

Starters

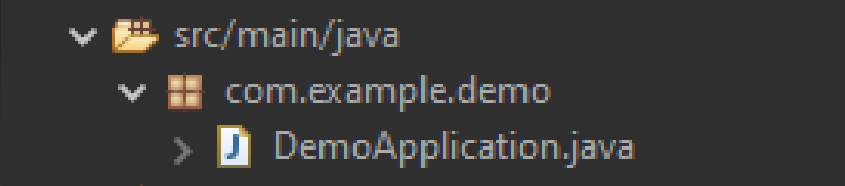
- Esistono 50+ starters che possiamo utilizzare per creare un progetto Spring Boot.
- Scelgo gli starters in base alle tecnologie e funzionalità che la mia applicazione deve avere.
- Altri:
 - spring-boot-starter-actuator
 - spring-boot-starter-web
 - spring-boot-starter-data-jpa
 - spring-boot-starter-security
 - ecc...

Struttura progetto



```
demo [boot]
├── src/main/java
│   └── com.example.demo
│       └── DemoApplication.java
├── src/main/resources
│   └── application.properties
├── src/test/java
│   └── com.example.demo
│       └── DemoApplicationTests.java
├── JRE System Library [JavaSE-17]
├── Maven Dependencies
├── src
├── target
├── HELP.md
├── mvnw
├── mvnw.cmd
└── pom.xml
```

Struttura progetto



```
▼ src/main/java
  ▼ com.example.demo
    > DemoApplication.java
```

- All'interno della cartella «src/main/java» ci saranno tutti i package e sottopackage dell'applicazione.
- All'interno del package che viene creato in automatico vanno creati manualmente ulteriori package che conterranno i componenti dell'applicazione: classi di configurazione, entities, services, controllers ecc...

Struttura progetto

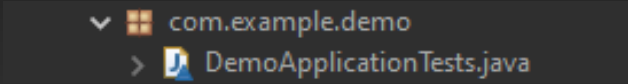
```
@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

}
```

- L'annotazione @SpringBootApplication incapsula le seguenti annotazioni:
- @Configuration -> Definire @Bean all'interno della classe annotata.
- @ComponentScan -> Creazione dei bean annotati con le annotazione stereotype presenti nel package e sotto package.
- @EnableAutoConfiguration -> Abilita l'auto configurazione di Spring Boot. Crea e configura ulteriori beans in base ai JAR presenti nel classpath.

Struttura progetto



```
▼ com.example.demo  
  > DemoApplicationTests.java
```

- All'interno della cartella «src/test/java» ci saranno tutti i package e sottopackage che conterranno le classi di test.
- Tali classi servono a testare i componenti dell'applicazione.

Struttura progetto

```
@SpringBootTest
public class DemoApplicationTests {

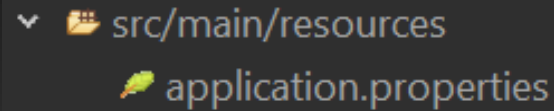
    void contextLoads() {

    }

}
```

- L'annotazione `@SpringBootTest` permette la creazione di un `ApplicationContext` che contiene i bean della nostra applicazione.
- Tramite l'annotazione `@Autowired` possiamo iniettare tali bean e testarli all'interno dell'ambiente di testing.
- La fase di testing è molto importante. Ci sono diverse metodologie, best practice e tecnologie che possiamo utilizzare.

Struttura progetto



```
▼ src/main/resources
  application.properties
```

- La cartella «src/main/resources» contiene le risorse dell'app.
- Di default viene generato un file chiamato «application.properties».
- Esiste anche l'estensione «application.yml»
- In questi file possiamo configurare Spring Boot.
- Spring Boot fornisce varie proprietà che possono essere configurate:
 - Server properties
 - Data properties
 - Mail properties
 - Security properties
 - Logging properties
 - ecc..